

Operation Code Rescue — Release Readiness Dossier

Master Index: Review, 50 Recommendations, UX Overhaul, and Technical Deep-Dives

Operation Code Rescue Prepared with Claude

June 25, 2026

Contents

0.1	How to use this dossier	1
0.1.1	Companion PDFs	1
0.1.2	Markdown references (same folder)	1
1	Executive summary of this pass	3
2	Recommended path to release	4

Operation Code Rescue is a first-person, post-apocalyptic survival-horror game that teaches real coding (Java, C, C++, Python, MATLAB) through terminal puzzles across a 465-city rescue campaign, built in Unreal Engine 5.7 on macOS. This dossier is the index to a release-readiness documentation set produced on June 25, 2026, covering a project review, fifty prioritized recommendations, a hands-on UX/UI design-system overhaul, a live Unreal editor bridge, and three in-depth technical chapters that chart the path to a fully functional release.

0.1 How to use this dossier

This master document links to four companion PDFs and several Markdown references that should sit in the **same folder** (`Documentation/improvement_pass_2026-06-25/`). In most PDF viewers (Preview, Acrobat) the links below open the sibling files directly.

0.1.1 Companion PDFs

- [Top 50 Recommendations](#) — prioritized, project-specific roadmap.
- [Character Animation Deep-Dive](#) — making complete, beautiful, unique animated characters in UE 5.7 on Apple Silicon.
- [Game Physics Deep-Dive](#) — establishing and maintaining proper Chaos physics throughout the experience.
- [World Development Deep-Dive](#) — building a complete, novel, release-quality world.

0.1.2 Markdown references (same folder)

- `PROJECT_STATE_REVIEW_2026-06-25.md` — full assessment of what exists and where the gaps are.
- `UX_OVERHAUL_GUIDE.md` — the design-system layer added this pass and its rollout plan.

- `UNREAL_LIVE_BRIDGE.md` — how Claude connects to and drives the Unreal Editor on your Mac.

Chapter 1

Executive summary of this pass

This pass delivered work on every front requested:

1. **Project review.** A grounded assessment: ~31,700 lines of C++, ~90 files, a full UMG UI suite, deep progression/accessibility systems, day/night + weather, achievements, leaderboards, a mod loader, 230 voiced briefings, and a thorough automated QA/release toolchain — a mature prototype whose remaining gap to release is concentrated in *character animation*, *physics*, *hand-crafted world art*, and *UX finish*.
2. **Top 50 recommendations.** Prioritized P0/P1/P2 across UX, characters, physics, world, combat, AI, audio, content, tech-debt, and release gates.
3. **UX/UI overhaul (implemented).** A centralized `CodeRescueUITheme` design system (semantic color, type scale, spacing, accessibility-aware styling) now drives the Main Menu, Pause, Settings, Death, Victory, and damage-feedback screens, with the HUD wired to honor high contrast, reduced motion, and text scaling live.
4. **Live Unreal bridge (implemented).** An editor-side Python bridge plus launcher lets Claude drive the editor on your Mac (inspect, screenshot, script, validate) across sessions.
5. **Three release-readiness deep-dives.** Character animation, physics, and world development — each tailored to this project, with concrete UE 5.7 workflows, Apple-Silicon caveats, phased solo-dev roadmaps, and verified citations.

Chapter 2

Recommended path to release

A pragmatic sequence that protects the strong foundation already built:

1. **Finish the UX rollout** (HUD + terminal) on the new design system.
2. **Build one polished vertical slice** of a single city: real animated characters (Animation chapter), Chaos physics for combat/ragdolls/throwables (Physics chapter), and an authored + PCG hybrid space (World chapter) — proving the whole production pipeline end-to-end.
3. **Scale the proven pipeline** to city families, migrate AI to Behavior Trees/StateTree, deepen audio, and split the 12k-line `CodeRescueGameMode.cpp`.
4. **Clear the release gates:** a human first-ten-minutes playtest, per-difficulty balance review, then Apple Developer signing/notarization and hosted Mac CI.

Full citations and references appear at the end of each deep-dive chapter (Character Animation, Game Physics, World Development), drawn primarily from official Epic Games documentation and reputable industry sources, verified during research.